Lecture 13

随机算法

13.1 基本思想与概念

前面设计的算法通常会明确规定下一步怎样选择，例如贪心算法每次选取当前最有利的对象，局部搜索则不断寻找能够改进当前解的邻居。本讲考虑另一种做法：在某些步骤中，让算法随机作出选择。这就得到了随机算法（randomized algorithm）。随机选择可能使同一个输入产生不同的执行过程，我们希望利用这种变化，改善运行时间，或者用较少的计算得到具有一定正确性保证的答案。

以快速排序为例，如果总用第一个元素作枢轴，已经排好序的数组反而会成为最坏输入。每次划分后，一边为空，另一边只少了一个元素，总时间需要 $\Theta(n^2)$。如果改为随机选择枢轴，有序数组可能被分成大小接近的两部分。当然枢轴可能偶尔选得不好仍会增加计算量，但每次都选到最坏枢轴的概率不高，后面将证明这种算法的期望时间为 $O(n \log n)$。

这里需要区分平均情况分析与随机算法的期望时间分析。两者虽然都在求平均运行时间，但随机性的来源不同。平均情况分析先假设输入服从某种概率分布，再对不同输入的运行时间取平均。例如，对于总选第一个元素作枢轴的快速排序，若各元素互不相同且各种排列等可能，平均时间也是 $O(n \log n)$，但遇到已经排好序的数组，仍然需要 $\Theta(n^2)$ 时间。随机快速排序则固定输入，对随机选择枢轴所产生的不同执行过程取平均。对于任意一个输入，包括已经排好序的数组，这一期望时间都是 $O(n \log n)$，因此无须假设输入的各种排列等可能。

下面我们给出随机算法的两种常见类型，称为 Las Vegas 与 Monte Carlo 算法。

定义 13.1. 1. **Las Vegas 算法**返回的答案总是正确的，随机选择影响的是运行时间。通常要求算法以概率 1 终止，并分析它的期望运行时间。

2. **Monte Carlo 算法**在规定的时间内返回答案，但允许以一定概率出错。除了运行时间，还要给出对每个输入都成立的错误概率上界。

随机快速排序属于 Las Vegas 算法，因为无论枢轴选得好不好，排序结果都正确。本讲补充材料中的素性检验则属于 Monte Carlo 算法，它只做有限轮测试，可能把某个合数判定为素数。

13.1.1 均匀随机整数

后面的算法经常需要从几个候选对象中等概率地选一个。我们用 RandomInt(a, b) 表示在整数 $a, a+1, \ldots, b$ 中均匀选取一个数。

假设我们能够得到独立的公平随机位，也就是概率各为 $1/2$ 的 0 和 1。怎样基于此产生所需的整数呢？令 $m = b - a + 1$，先用 $q = \lceil \log_2 m \rceil$ 个随机位组成一个二进制数 u，这样 $0, \ldots, 2^q - 1$ 等可能出现。若 $u < m$，返回 $a + u$，否则重新抽取，直到落入所需范围。

不难理解，当 $u < m$ 时，u 取 $0, \ldots, m-1$ 的概率相同，所以返回值均匀。接下来需要计算平均要尝试多少轮，为此先证明下面的引理。

> **引理 13.2.** 独立重复成功概率为 $p > 0$ 的试验，直到第一次成功。记试验次数为 T，则 $\mathbb{E}[T] = 1/p$。如果各次试验不独立，但无论此前怎样失败，下一次成功的条件概率都至少为 p，仍有 $\mathbb{E}[T] \leqslant 1/p$。

证明. 第 k 次才首次成功，意味着前 $k-1$ 次都失败、第 k 次成功。因此，独立试验时 $\Pr(T = k) = (1-p)^{k-1}p$。由期望的定义可得

$$\mathbb{E}[T] = \sum_{k=1}^{\infty} k(1-p)^{k-1}p = p\sum_{j=1}^{\infty}\sum_{k=j}^{\infty}(1-p)^{k-1} = \sum_{j=1}^{\infty}(1-p)^{j-1} = \frac{1}{p}.$$

若每次成功的条件概率至少为 p，则前 $j-1$ 次全部失败的概率至多为 $(1-p)^{j-1}$，则有

$$\mathbb{E}[T] = \sum_{k=1}^{\infty} k\Pr(T=k) = \sum_{j=1}^{\infty}\sum_{k=j}^{\infty}\Pr(T=k) = \sum_{j=1}^{\infty}\Pr(T \geqslant j) \leqslant \sum_{j=1}^{\infty}(1-p)^{j-1} = \frac{1}{p}.$$

□

现在回到随机整数的生成过程。每轮共有 2^q 个等可能的结果，其中 $0, \ldots, m-1$ 这 m 个数可以直接使用，其余结果需要重新抽取。由于 $q = \lceil \log_2 m \rceil$，当 $m > 1$ 时有 $2^q < 2m$，所以每轮成功的概率 $m/2^q > 1/2$。由引理 13.2，平均不到两轮就能得到所需整数。当 $m = 1$ 时，直接返回唯一的整数 a 即可，无须抽样。

后面分析算法时，通常把一次 RandomInt(a, b) 调用按期望 $O(1)$ 时间计算。这里假设所需整数能存入一个机器字，并且生成一个机器字的随机位、比较和加减都只需常数时间。这样，上面每轮抽样只需常数时间，平均又不到两轮，便得到了这一时间界。素性检验处理的整数可能超过一个机器字，其运算时间会在相应小节中另行分析。

13.1.2 伪随机数

上一节假设我们能够不断取得独立的随机位。在实际编程中，通常会调用随机数函数来完成抽样。这类函数常用一个初始值和固定的计算规则生成数列，再用其中的数作随机选择。其中初始值称为种子，生成数列的程序称为**伪随机数发生器（pseudorandom number generator）**。

例如，线性同余发生器选定整数参数 a, c, M（$M > 0$），从种子 x_0 出发，按递推式

$$x_{i+1} = (ax_i + c) \bmod M$$

依次生成后续的数。取 $M = 11, a = 7, c = 0, x_0 = 1$，第一项是 7，第二项是 $7 \times 7 \bmod 11 = 5$，继续计算得到 $7, 5, 2, 3, 10, 4, 6, 9, 8, 1, \ldots$。第十项又回到了种子 1，此后便会重复这十个数。

种子和计算规则确定后，整个序列就确定了。因此，使用相同的种子可以重复同一次实验，便于调试和比较算法。不过，即使各个数在一个周期中出现得同样频繁，也不能说明它们相互独立。例如，上面的序列一旦出现 7，下一项就一定是 5。本讲的概率分析仍以独立随机抽样为前提，但使用伪随机数实现时，需要注意发生器产生的相关性是否会影响算法。

密码应用要求攻击者即使知道生成规则和已经输出的数，仍然难以预测后续结果。上面的线性同余发生器直接输出 x_i，而参数 a, c, M 一旦已知，攻击者就能代入递推式算出 x_{i+1}，甚至整个后续序列，无须知道最初的种子。因此，这种发生器不适合直接用于密码应用。

13.2　雇佣问题与随机排列

13.2.1　雇佣次数的期望

我们要招聘一位员工，假设共有 n 位应聘者依次来面试，每人的能力都不同。为了始终保留目前见过的最优秀的人，每当新应聘者比现任更优秀时，就辞退现任并雇佣新人。面试一个人需要成本 c_i，每次雇佣需要成本 c_h，而更换员工通常远比面试成本高，即 c_h 远大于 c_i。

无论应聘者按什么顺序到来，都需要面试 n 次，因此面试成本固定为 nc_i。真正影响总成本的是更换了多少次人选。我们来看一个简单的例子。如果用 $1,\dots,8$ 表示从低到高的能力，图 13.1 中的到达顺序会使我们依次雇佣能力为 $3,5,7,8$ 的四个人。若应聘者恰好按能力递增的顺序到来，每个人都会比现任优秀，就要雇佣 n 次。

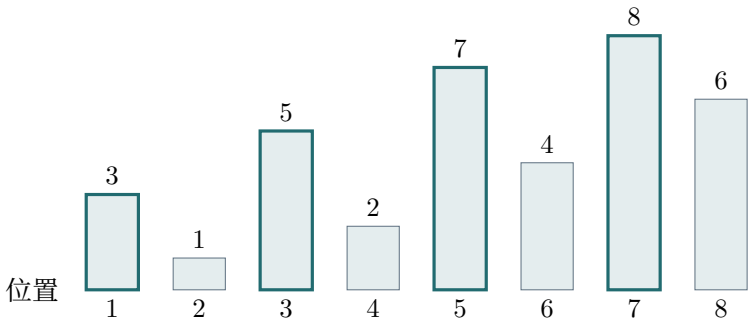


图 13.1　按所示顺序面试时，粗框中的 $3,5,7,8$ 都比此前的人优秀，因此会被依次雇佣。

下面我们分析随机顺序下的期望雇佣次数。假设应聘者以均匀随机的顺序到来。第 i 人会被雇佣，当且仅当他是前 i 人中最优秀的。无论这 i 人具体是谁，最优秀者出现在这 i 个位置中的概率都相同，所以第 i 人被雇佣的概率为 $1/i$。令 I_i 在第 i 人被雇佣时取 1，否则取 0，总雇佣次数便是 $H=\sum_{i=1}^{n} I_i$。利用期望的线性性，得到 $\mathbb{E}[H]=\sum_{i=1}^{n}\mathbb{E}[I_i]=\sum_{i=1}^{n}\frac{1}{i}=H_n$，其中 H_n 称为第 n 个调和数，并约定 $H_0=0$。对 $n\geqslant 1$，将和式与 $1/x$ 的积分比较，可得 $\ln(n+1)\leqslant H_n\leqslant 1+\ln n$。因此，随机顺序下的期望雇佣次数只有 $\Theta(\log n)$，远少于最坏情况下的 n 次。

如果面试顺序可以由我们安排，就不必假设应聘者恰好随机到来。我们只需先将名单均匀打乱，再进行面试。这样，无论原名单按什么顺序排列，都有前述期望保证。下面说明如何在 $O(n)$ 的期望时间内完成均匀打乱。

13.2.2　Fisher–Yates 洗牌

要把名单均匀打乱，可以先从全部 n 人中任选一人，放在最后一个位置，再从剩下的 $n-1$ 人中任选一人，放在倒数第二个位置，依次进行。这种方法称为 Fisher–Yates 洗牌。固定任意一个排列，考察算法恰好得到它的概率。最后一个位置选对的概率为 $1/n$。在已经选对的条件下，倒数第二个位置选对的概率为 $1/(n-1)$。依次相乘，整个排列出现的概率为 $\frac{1}{n}\cdot\frac{1}{n-1}\cdots\frac{1}{2}=\frac{1}{n!}$。每个排列都满足这个结论，因此算法确实得到均匀随机排列，且只需 $O(n)$ 时间。

另一种做法是为每个元素独立地从 $1,\dots,n^3$ 中均匀抽取一个整数作为优先级，再按优先级排序。如果优先级互不相同，各种相对次序便等可能出现。但优先级取自有限的整数范围，可能出现相同值。例如两个元素的优先级相同时，总让原来靠前的元素排在前面，就会偏向原来的顺序。为保持均匀性，可以在发现重复优先级时重新生成整组优先级。

为了估计出现重复优先级的概率，我们使用下面的**并集界（union bound）**。

引理 13.3. 对同一概率空间中的任意事件 $A_1, \ldots, A_m$，都有

$$\Pr\left(\bigcup_{i=1}^{m} A_i\right) \leqslant \sum_{i=1}^{m} \Pr(A_i).$$

即至少一个事件发生的概率，不超过各事件发生概率之和。

固定任意两个元素，第一个元素的优先级确定后，第二个元素有 n^3 个等可能的选择，其中只有一个与第一个元素的优先级相同，所以两者相同的概率为 $1/n^3$。共有 $\binom{n}{2}$ 对元素，由引理 13.3，至少出现一对重复优先级的概率至多为 $\frac{\binom{n}{2}}{n^3} = \frac{n-1}{2n^2} < \frac{1}{2n}$。因此每轮成功概率超过 $1/2$，引理 13.2 表明期望尝试次数少于两次。每轮使用比较排序需要 $O(n \log n)$ 时间，所以总期望时间为 $O(n \log n)$。

13.3 秘书问题

雇佣问题允许我们不断更换员工，但有些招聘只允许作出一次选择。假设应聘者人数 n，能力互不相同，且到达顺序为均匀随机排列。我们要求拒绝的应聘者不能再召回，而一旦雇佣某个人，招聘就立即结束。我们只知道已经面试过的人的相对能力，仍希望尽可能选中所有人中最优秀的那位，这就是**秘书问题（secretary problem）**。

一种自然的策略是先看一部分人，了解大致的能力水平，再以此为依据进行后续选择。具体地，我们将前 k 人全部拒绝，但记住其中的最优秀者，此后遇到第一个比他更优秀的人，就立即雇佣。如果直到最后也没有遇到，就雇佣最后一个人。事实上这一思想与**多臂老虎机（multi-armed bandit）**问题中的**探索与利用（exploration and exploitation）权衡**类似。前 k 人用于探索整体的能力，然后我们利用探索得到的信息作出选择。如果观察的人太少，容易过早接受普通人，而观察的人太多，又可能在观察阶段就拒绝了最优秀者，因此关键在于选择合适的 k，也就是权衡探索与利用。

例如，按图 13.2 的顺序面试，取 $k = 3$ 时，前三人的最高能力是 5，之后首先超过它的是第五人 7。算法因此选中 7，错过了后来才出现的 8。

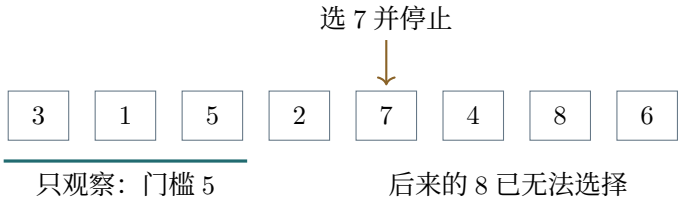


图 13.2 先观察前三人，再接受第一个能力超过 5 的应聘者，因此选择 7 后就停止。

接下来分析成功选中最优秀者的概率。先考虑 $1 \leqslant k < n$。如果最优秀者在前 k 人中，就会在观察阶段被拒绝。假设他出现在第 $i > k$ 个位置，要成功选中他，算法必须在此前一直没有接受其他人，即要求前 $i-1$ 人中最优秀者位于前 k 人中。由于所有人中最优秀者出现在第 i 位的概率为 $1/n$，确定他的位置后，其余人的相对次序仍然均匀随机，所以前 $i-1$ 人中最优秀者位于前 k 人中的概率为

$k/(i-1)$。按 i 求和，得到先拒绝前 k 人的情况下成功选中最优秀者的概率为

$$p_n(k) = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}, \qquad 1 \leqslant k < n. \tag{13.1}$$

其中求和从 $k+1$ 开始是因为如果 $i \leqslant k$，最优秀者在观察阶段就被拒绝了。此外，若 $k = n$，算法强制选择最后一人，因此成功概率为 $p_n(n) = 1/n$。若 $k = 0$，算法直接接受第一人，成功概率为 $p_n(0) = 1/n$。

一个自然的问题是，当人数很大时，最佳观察人数大约是多少？对 $k > 1$，用积分估计式 (13.1) 中的和式，得到

$$\frac{k}{n}\ln\frac{n}{k} \leqslant p_n(k) \leqslant \frac{k}{n}\ln\frac{n-1}{k-1}.$$

如果当 n 很大时，k/n 趋于固定比例 $\alpha \in (0,1)$，上式两端都趋于 $\alpha\ln(1/\alpha)$，其在 $\alpha = 1/e$ 处取得最大值 $1/e$。这说明先观察约 n/e 人，再接受第一个超过观察阶段最佳人选的应聘者，成功概率会随 n 增大而趋于 $1/e$，约为 36.8%。

13.4 随机选择与快速排序

第七讲的选择算法和快速排序都要选取一个基准值，将数组分成小于、等于和大于它的三组，这个基准值也称为枢轴（pivot）。两种算法的效率都与枢轴的选择有关。如果每次都选中最小值或最大值，划分就只能排除很少的元素；若能把左右两组分得比较均匀，后续需要处理的数组就会明显变短。本节尝试均匀随机选取枢轴，分析这样做能否得到较好的期望时间。

13.4.1 随机选择

先来看第七讲讨论过的选择问题，即给定一个数组和整数 k，找出数组中第 k 小的元素。当时为了保证线性时间，我们通过寻找中位数的中位数来选取枢轴，但这一步本身也需要分组和递归。如果每次直接随机选一个元素作枢轴，就能省去这些操作，得到更简单的随机选择算法。

具体而言，每次从当前数组中均匀随机选出一个枢轴 v，将元素分成小于、等于和大于 v 的三组。假设前两组分别有 ℓ 和 e 个元素，则若 $\ell < k \leqslant \ell + e$，答案就是 v，若 $k \leqslant \ell$，答案在小于 v 的一组中，若 $k > \ell + e$，答案就在大于 v 的一组中。进入小于 v 的一组时，仍然寻找其中第 k 小的元素。进入大于 v 的一组时，已经排除了 $\ell + e$ 个更靠前的元素，所以改为寻找其中第 $k - \ell - e$ 小的元素。每次都在保留的一组中重新随机选择枢轴，继续划分，直到答案等于枢轴，或者只剩一个元素时直接返回。

不难理解，划分 m 个元素需要 $\Theta(m)$ 时间。然而随机选择仍有可能连续选到很差的枢轴。例如，要找最大值，却每次都选中当前最小值，就要依次处理 $n, n-1, \ldots, 1$ 个元素，总时间为 $\Theta(n^2)$。当然，由于枢轴是随机选取的，每次都选中当前最小值的概率较小。接下来我们就来分析期望时间复杂度。

对当前的 m 个元素，若一次划分后左右两组都至多有 $3m/4$ 个元素，就称为一次**好划分**。无论答案在哪一侧，下一次需要处理的元素都至多为原来的 $3/4$。这样的枢轴并不少，排序后位于中间一半的元素都能满足要求。具体来说，先将这 m 个元素按值排列，相同值任意确定先后次序。若枢轴在其中的名次 s 满足 $m/4 \leqslant s \leqslant 3m/4 + 1$，则严格小于它的元素至多有 $s - 1 \leqslant 3m/4$ 个，严格大于它的元素至多有 $m - s \leqslant 3m/4$ 个。满足条件的整数名次至少有 $m/2$ 个，因此每次均匀随机选择枢轴，得到好划分的概率至少为 $1/2$。

为了计算总时间，我们先看元素数从 n 降到至多 $3n/4$ 之前，要进行多少次划分。由于得到好划分的概率至少为 $1/2$，所以期望至多尝试两次划分。每次处理的元素不超过 n，这部分的期望时间便至多为 $2cn$，其中 c 是划分所需时间的常数系数。元素数降到至多 $3n/4$ 后，再用相同的办法分析降到至多 $n(3/4)^2$ 所需的时间，得到上界 $2cn(3/4)$。依次进行下去，在元素数满足 $n(3/4)^{j+1} < m \leqslant n(3/4)^j$ 的阶段，划分期望至多花费 $2cn(3/4)^j$ 时间。如果某次划分使元素数减少得更多，跳过了某个阶段，该阶段就不产生计算量，因此这样计算得到的是时间复杂度的上界。将各阶段的期望时间相加，得到

$$\mathbb{E}[T(n)] \leqslant \sum_{j=0}^{\infty} 2cn(3/4)^j = 8cn = O(n).$$

因此，对任意固定的输入和合法的 k，随机选择的期望时间都是 $O(n)$。与第七讲的确定性选择相比，它省去了专门寻找枢轴的过程，但线性时间的保证也由最坏情况变成了期望意义下的保证。

13.4.2 随机快速排序

在选择问题中，我们只关心第 k 小的元素，因此划分后可以舍弃答案所在组以外的两组。如果要把整个数组排好序，小于和大于枢轴的两组都需要继续排序。具体来说，每次从当前区间中均匀随机选一个元素作枢轴，完成三路划分后，分别递归排序小于枢轴和大于枢轴的两组，等于枢轴的一组不必再处理。区间为空或只有一个元素时，直接结束这次调用。这就是随机快速排序。

下面分析时间复杂度。先固定一个元素，计算它参与划分的次数的期望。每遇到一次好划分，它所在的区间就至少缩短四分之一。如果它进入等值组，之后便不再参与划分。按照上一小节的方法，将区间长度依次分为 n 到 $3n/4$、$3n/4$ 到 $n(3/4)^2$ 等阶段，每个阶段期望至多参与两次划分。区间长度从 n 缩短到常数只需 $O(\log n)$ 个阶段，所以每个元素期望参与 $O(\log n)$ 次划分。一次划分花费的时间与其中的元素数成正比，因此可以把这一时间平均分给参与的元素，每个元素只需承担常数时间。再将全部 n 个元素承担的时间相加，就得到期望总时间 $O(n \log n)$。

当各元素互异时，还可以用指示变量准确计算比较次数。将排序后的元素记为 $z_1 < \cdots < z_n$，考虑 z_i 和 z_j（$i < j$）是否会相互比较。快速排序只将枢轴与当前区间中的元素比较，因此要让这两个元素比较，其中一个必须先成为枢轴，而另一个仍与它处在同一区间。

是否会比较 z_i 和 z_j，取决于 $z_i, z_{i+1}, \ldots, z_j$ 中哪一个元素最先被选作枢轴。若选中的是 z_i 或 z_j，两者就会比较，若选中的是中间的某个元素，它就将 z_i, z_j 分到两侧，两者以后都不会再比较。如果此前选中的枢轴小于 z_i 或大于 z_j，那么 $z_i, z_{i+1}, \ldots, z_j$ 会全部进入同一组。因此，在这些元素中首次选出枢轴之前，z_i 和 z_j 始终在同一组中，尚未相互比较，也没有被分开。由于每次均匀选取枢轴，这 $j - i + 1$ 个元素中每一个都有相同的概率最先被选中，因此两者比较的概率为 $2/(j - i + 1)$。

令 I_{ij} 在 z_i, z_j 发生比较时取 1，否则取 0。每对不同元素至多比较一次，所以它们之间的比较总数为 $C = \sum_{i<j} I_{ij}$，从而

$$\mathbb{E}[C] = \sum_{1 \leqslant i < j \leqslant n} \frac{2}{j - i + 1} = 2\sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2(n+1)\sum_{d=1}^{n-1} \frac{1}{d+1} - 2(n-1) = 2(n+1)H_n - 4n.$$

其中 $H_n = \sum_{i=1}^{n} 1/i = \Theta(\log n)$，所以比较次数的期望为 $\Theta(n \log n)$。

13.4.3 限制划分比例的快速排序

前面的快速排序无论划分结果如何，都会继续递归。另一种做法是先检查左右两组的长度，只有它们都不超过当前元素数的 $3/4$ 时才继续递归，否则就在当前整个区间内重新选择划分。

记 $T(n)$ 为长度为 n 的各个输入中最大的期望运行时间。每次调用期望至多尝试两次划分，递归前的期望时间为 $O(n)$。划分成功后，左右两组的元素数 a, b 都至多为 $3n/4$，且至少有一个元素进入中间组，所以 $a + b \leqslant n - 1$。因此有

$$T(n) \leqslant \max_{\substack{a,b\geqslant 0,\ a+b\leqslant n-1\\ a,b\leqslant 3n/4}} \{T(a) + T(b)\} + O(n),$$

其中 $T(0) = T(1) = O(1)$。用第七讲的递归树方法分析，每层元素总数至多为 n，故每层的时间为 $O(n)$。每次递归后元素数至多为原来的 $3/4$，所以共有 $O(\log n)$ 层，由此得到期望时间 $O(n \log n)$。

需要注意，这一算法可能连续任意多次抽到不合格的枢轴，所以没有确定的最坏运行时间上界。不过，连续失败 t 次的概率至多为 2^{-t}，随 t 增大趋于 0，因此每次调用都以概率 1 得到好划分。递归调用总数有限，整个算法也以概率 1 结束。

13.5 球入箱问题

如果要将 m 项任务分给 n 台机器，其中每项任务独立地随机选择一台机器，那么平均来看，每台机器应该分到 m/n 项任务，但实际分配可能很不均衡。我们希望进一步知道，有多少机器会被用到，任务是否容易集中到同一台机器，以及最忙的机器会有多少任务。

这一问题可以描述为一般的**球入箱（balls into bins）**模型：将 m 个球独立、均匀地投入 $n \geqslant 1$ 个有编号且不限制容量的箱子中。事实上，散列中的键与桶也可用这个模型表示，其中多个球落入同一个箱子就对应散列冲突。

13.5.1 球的分布与碰撞

投入这 m 个球后，我们先计算三个数量的期望：一个指定箱子中的球数、落入同一箱子的球对数，以及至少装有一个球的箱子数。它们分别说明每个箱子平均装多少球，同箱的情况有多常见，以及有多少箱子会被用到。

先求一个指定箱子中的期望球数。记箱子 j 中的球数为 L_j，也称为该箱子的负载。对每个球 i，若它落入箱子 j，就令 $I_{ij} = 1$，否则令 $I_{ij} = 0$。由于每个球落入这个箱子的概率都是 $1/n$，将这些指示变量相加并取期望，便有 $\mathbb{E}[L_j] = \sum_{i=1}^{m} \mathbb{E}[I_{ij}] = m/n$。

两个球落入同一个箱子时，称它们构成一对碰撞。对每对球 i, j，其中 $i < j$，若它们落入同一个箱子，就令 $J_{ij} = 1$，否则令 $J_{ij} = 0$。第一个球可以落在任意位置，第二个球与它同箱的概率为 $1/n$，所以 $\mathbb{E}[J_{ij}] = 1/n$。将所有球对的指示变量相加，得到 $\mathbb{E}[C] = \sum_{1\leqslant i<j\leqslant m} \mathbb{E}[J_{ij}] = \frac{1}{n}\binom{m}{2}$。若只关心是否发生过碰撞，对所有球对使用并集界，得到 $\Pr(C \geqslant 1) \leqslant \frac{1}{n}\binom{m}{2}$。

再看有多少箱子被使用。对每个箱子 j，若它至少装有一个球，就令 $K_j = 1$，否则令 $K_j = 0$。一个箱子为空，要求每个球都没有落入它，概率为 $(1 - 1/n)^m$，所以 $\mathbb{E}[K_j] = 1 - (1 - 1/n)^m$。将各箱子的指示变量相加，得到 $\mathbb{E}[U] = \sum_{j=1}^{n} \mathbb{E}[K_j] = n\big(1 - (1 - 1/n)^m\big)$。换个角度理解，如果每个箱子只接受第一个到来的球，后来到达的球全部拒绝，那么接受的球数恰好是 U。

13.5.2 生日悖论与优惠券收集问题

一个自然的问题是，在投入多少球后，碰撞会比较常见？若 $m > n$，则必然发生碰撞，因此我们考虑 $m \leqslant n$ 的情况。可以先计算完全没有碰撞的概率：第一个球可以随意选择箱子，随后第二个球

要避开第一个已占用的箱子，第三个球要避开前两个已占用的箱子，以此类推。仍记 C 表示碰撞数，得到

$$\Pr(C=0)=\prod_{i=0}^{m-1}\left(1-\frac{i}{n}\right)\leqslant\exp\left(-\frac{m(m-1)}{2n}\right).$$

不等式使用了 $1-x\leqslant e^{-x}$。因此，只要 m 达到 $\sqrt{n}$ 的量级，发生碰撞的概率就能达到常数，而此时大多数箱子甚至还没有被占用。

这也被称为**生日悖论（birthday paradox）**：在一个班级中，人数只要达到 23，就有超过 50% 的概率至少有两人同一天生日。这是因为我们可以把一年中的 365 天看成箱子，把每个人的生日看成球，并假设各人生日独立且每天等可能。代入上面的乘积可得，22 人中至少两人同一天生日的概率约为 0.476，23 人时就约为 0.507。

如果希望每个箱子都至少收到一个球，又需要投多少次呢？当已有 j 个箱子非空时，下一个球落入新箱子的概率为 $(n-j)/n$，所以等到下一个新箱子被占用，引理 13.2 表明期望需要 $n/(n-j)$ 次。将这些等待次数相加，得到

$$\mathbb{E}[T]=\sum_{j=0}^{n-1}\frac{n}{n-j}=nH_n.$$

特别地，只剩最后一个空箱子时，还要期望再投 n 次。这个模型也称为**优惠券收集问题（coupon collector problem）**：每次随机得到一种优惠券，我们希望收齐全部种类。这个问题的一个特点是，越到最后越容易抽到已经有的种类。

13.5.3 最大负载

令 $M=\max_j L_j$ 表示最满箱子的负载。要使 $M\geqslant k$，必须有某个箱子收到至少 k 个球，也就必定存在一组 k 个球全部进入该箱子。固定一个箱子和一组 k 个球，这组球全部落入该箱子的概率为 n^{-k}。由于有 n 个箱子和 $\binom{m}{k}$ 种球组，对所有选择使用并集界，得到

$$\Pr(M\geqslant k)\leqslant\min\left\{1,n\binom{m}{k}n^{-k}\right\}\leqslant\min\left\{1,n\left(\frac{em}{nk}\right)^k\right\},\qquad 1\leqslant k\leqslant m.\tag{13.2}$$

第二个不等式使用 $\binom{m}{k}\leqslant m^k/k!$ 和 $k!\geqslant(k/e)^k$，后者可由下式得到：$\ln k!=\sum_{i=1}^{k}\ln i\geqslant\int_1^k\ln x\,dx=k\ln k-k+1$。如果 $k>m$，球的总数都不足 k，所求概率自然为 0。

以 $m=n$ 为例，每个指定箱子的期望负载只有 1，但最大负载需要更大的上界。取 $k=\left\lceil\frac{3\ln n}{\ln\ln n}\right\rceil$。当 n 足够大时，$\ln k=\ln\ln n-\ln\ln\ln n+O(1)$，所以式 (13.2) 中 $n(e/k)^k$ 的对数为 $\ln n+k(1-\ln k)=(-2+o(1))\ln n$。它最终不超过 $-\ln n$，因而 $\Pr(M\geqslant k)\leqslant 1/n$。也就是说，以至少 $1-1/n$ 的概率，最大负载不会达到上述 k，从而得到高概率上界 $O(\ln n/\ln\ln n)$。

13.6 数据流抽样

13.6.1 抽取一个元素

如果我们希望从一份名单中随机抽取一个人，最简单的办法是先把名单全部读入内存，然后均匀选择一个数组下标。但如果数据不断到来，我们事先不知道共有多少项，也没有空间保存全部数据，就无法等读完后再随机选一个位置回头取数。我们的问题是，怎样在只读一遍数据的前提下，仍能保证每项数据被选中的概率相同？

一种简单的想法是，先保留第一项，当读到第二项时，以 $1/2$ 的概率改为保留第二项，当读到第三项时，以 $1/3$ 的概率改为保留第三项，以此类推。一般地，第 i 项到来时，以 $1/i$ 的概率用它替换当前样本，否则继续保留旧样本。

假设数据流最终有 n 项。第 j 项要成为最后留下的样本，需要在到来时被选中，并在后续每次更新中都没有被替换。因此它最终留下的概率为

$$\frac{1}{j}\prod_{i=j+1}^{n}\left(1-\frac{1}{i}\right)=\frac{1}{j}\cdot\frac{j}{j+1}\cdots\frac{n-1}{n}=\frac{1}{n}.$$

于是，上述算法能保证每个位置上的数据最终被选中的概率都相同。需要注意的是，不同位置上的数据可能相同，但我们认为它们是不同的样本，下一小节抽取 k 个元素时也会区分位置。

13.6.2 抽取 k 个元素

上一节只抽取一项数据，现在希望从数据流中均匀抽取 k 项，并且每项数据仍然只能读取一次。具体而言，若数据流共有 $n\geqslant k$ 项，我们要求每个含 k 项的子集都以 $1/\binom{n}{k}$ 的概率被选中。

为此，准备一个容量为 k 的数组 R，先保存前 k 项。第 $i>k$ 项到来时，以 k/i 的概率接受它，如果接受，就从当前保存的 k 项中均匀选一项，用新数据替换。这两步可以合成一次抽样：从 $1,\dots,i$ 中均匀选择整数 j，若 $j\leqslant k$，就替换 $R[j]$，否则保持原样，每次抽样都独立进行。这样读完整个数据流后，R 中的 k 项就是所求样本。这种方法称为**蓄水池抽样（reservoir sampling）**。

> **定理 13.4.** 处理完前 $i\geqslant k$ 项数据后，上述算法得到的每个 k 元位置子集的概率都为 $1/\binom{i}{k}$。

证明. $k=0$ 时只会返回空集。以下设 $k\geqslant 1$，对已读数据项数 i 作归纳。读完前 k 项时只有一种选择，结论成立。假设读完前 $i-1$ 项时结论成立，考虑前 i 项中的任意一个 k 元位置集合 S。若 $i\notin S$，最终得到 S，就要求旧样本已经是 S，并且本轮不接受新数据。因此概率为 $\frac{1}{\binom{i-1}{k}}\left(1-\frac{k}{i}\right)=\frac{1}{\binom{i}{k}}$。若 $i\in S$，旧样本必须包含 S 中其余的 $k-1$ 项，以及一个不属于 S 的旧位置 x。前 $i-1$ 个位置中，这样的 x 有 $i-k$ 种选择。对每种旧样本，本轮抽中 x 所在数组位置的概率都是 $1/i$，所以总概率为 $(i-k)\frac{1}{\binom{i-1}{k}}\frac{1}{i}=\frac{1}{\binom{i}{k}}$。两种情况下都得到同一个概率，归纳完成。 □

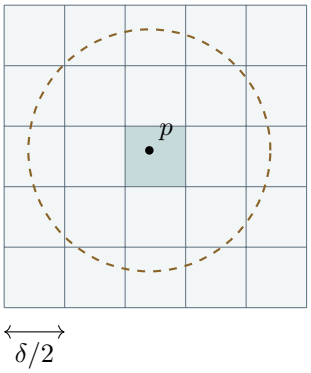
13.7 最近点对问题

给定平面上的 n 个点，我们希望找出距离最近的两个点。第七讲已经用分治法将时间复杂度降到了 $O(n\log n)$。这一节进一步考虑能否利用随机化减少计算量，得到期望线性时间的算法。

我们先换一种处理方式，从两个点开始，每次加入一个新点，并求出目前这些点中的最近点对。不过，如果每加入一个点，都将它与全部旧点比较，仍然要计算 $\Theta(n^2)$ 次距离。设目前的最小距离为 $\delta>0$，距离新点不小于 δ 的旧点都不会改善答案。因此，我们希望直接找到新点附近的旧点，省去对远处各点的检查。

为此，把平面划分成边长为 $\delta/2$ 的正方形网格。一个格子的对角线长为 $\delta/\sqrt{2}<\delta$，而任意两个旧点之间的距离都至少为 δ，所以每个格子至多包含一个旧点。我们只需用一张散列表保存非空格子，以格子的编号为键、其中的点为值，就能按编号查找某个格子中的点。具体地，点 (x,y) 所在格子的编号取为 $K_\delta(x,y)=\left(\left\lfloor\frac{2x}{\delta}\right\rfloor,\left\lfloor\frac{2y}{\delta}\right\rfloor\right)$。这相当于从原点开始，分别给横向和纵向的格子编号。我们默认落在格线上的点归入右侧或上侧的格子，因此每个点都有唯一的格号。

设新点所在格子的编号为 (s,t)。如果一个旧点所在格子的横向编号与 s 相差至少 3，两个点在横向上就相隔超过 δ，不可能组成更近的点对，纵向也是如此。因此，只需检查横、纵编号分别为 $s-2,\ldots,s+2$ 和 $t-2,\ldots,t+2$ 的 25 个格子。每格至多有一个旧点，所以每次只需计算至多 25 个距离。图 13.3 展示了这个范围。



横、纵格号各相差至多 2
共检查 25 格

图 13.3 虚线圆半径为 δ，只有圆内旧点可能与新点 p 组成更近的点对，而这些点都在图中 25 个格子内。

检查后，如果最小距离没有改变，就把新点及其格号加入散列表。它所在的格子一定为空，否则它与同格旧点的距离就会小于 δ。如果最小距离降为 δ'，则把网格边长改成 $\delta'/2$，并重建散列表。

总结一下完整的算法：先将输入点均匀打乱，用前两个点确定最初的最近点对、距离和网格，再按上述方法逐个加入其余点，判断是否有更近的点对，如果有则更新最小距离和对应的点对，并重建网格和散列表。然而如果每次加入新点时最小距离都下降，重建散列表仍可能花费平方时间。

事实上，算法开始时将点随机打乱，正是为了控制期望时间。接下来我们就来分析算法的期望时间。加入第 i 个点时，重建需要重新处理 i 个点。我们来计算这一步发生重建的概率。为处理最近点对并列的情况，先约定在随机打乱之前给各点固定编号，打乱后编号保持不变。现在固定前 i 个点组成的集合 S，在其中选定一对最近点 a,b。若有多对并列，就将每对的两个编号按从小到大排列，先比较第一个编号，再比较第二个，选编号最小的一对。这样，只要集合 S 相同，选出的点对就相同，与这些点的加入顺序无关。由于输入已经均匀打乱，S 中每个点成为第 i 个加入的点的概率都是 $1/i$。只有 a 或 b 最后加入时才可能重建，因此重建的概率至多为 $2/i$。无论前 i 个点组成哪个集合，这个上界都成立，所以第 i 步重建的概率也至多为 $2/i$。

现在把各步的工作量相加。用 I_i 表示第 i 步是否重建，重建时取 1，否则取 0，则该步因重建而处理的点数为 iI_i。由期望的线性性，全部重建处理的点数期望至多为

$$\mathbb{E}\left[\sum_{i=3}^{n} iI_i\right] = \sum_{i=3}^{n} i\Pr(I_i=1) \leqslant \sum_{i=3}^{n} i\frac{2}{i} = 2(n-2) = O(n).$$

再加上最初打乱点的顺序，以及每一步检查附近格子、插入新点的工作，算法总共的期望时间是 $O(n)$。

13.8 补充材料

13.8.1 限制运行时间与重复试验

前面分析了多种随机算法的期望运行时间，但实际使用时，我们有时还希望规定程序最多运行多久。例如，一个 Las Vegas 算法的期望时间至多为 $f(n)$，能否让它在规定时间内给出答案，并估计没有完成的概率？

记算法的运行时间为 T，先考虑运行超过 $cf(n)$ 的概率，其中 $c > 1$。由马尔可夫不等式，有

$$\Pr(T > cf(n)) \leqslant \frac{\mathbb{E}[T]}{cf(n)} \leqslant \frac{1}{c}.$$

例如，允许算法运行 $2f(n)$ 的时间，它仍未完成的概率至多为 $1/2$。于是，我们可以给每次尝试设定 $2f(n)$ 的时间上限。如果在时限内完成，就返回答案，否则中止本次运行，从头再试，并重新作出独立的随机选择。固定输入后，每次尝试超时的概率都至多为 $1/2$，所以连续 r 次都超时的概率至多为 2^{-r}。最多尝试 r 次，就能把总运行时间限制在 $2rf(n)$ 以内，并以至少 $1 - 2^{-r}$ 的概率得到答案，若全部超时，就报告失败。

前面是在算法超时后重新运行。有时算法能够很快结束，但得到的结果未必满足题目要求。如果我们有办法准确检查这个结果，就可以先运行算法，再检查结果，满足要求便返回，否则重新作出独立的随机选择，再试一次。由于每次返回前都经过检查，最终给出的答案一定正确。

这样做需要多长时间，取决于每次尝试成功的概率。假设每次独立尝试成功的概率至少为 $p > 0$，那么由引理 13.2，直到成功为止，期望至多尝试 $1/p$ 次。如果每次运行算法并检查结果共需 $O(g(n))$ 时间，总期望时间就是 $O(g(n)/p)$。因此，只要能够准确检查结果，就能把每次可能失败的随机尝试，变成答案始终正确的 Las Vegas 算法。如果无法检查结果是否正确，我们就不知道该返回答案还是继续尝试，上面的做法也就无法直接使用。

13.8.2　跳表

前面介绍的 AVL 树和红黑树都能高效地完成查找、插入和删除。这一节介绍另一种支持这些操作的数据结构——跳表。它在链表上增加多层指针，并用随机选择决定各层包含哪些结点，使这些操作的期望时间达到 $O(\log n)$。

我们先从普通链表出发。把结点按键值递增排列，就得到有序链表，但查找时仍可能需要从表头逐个比较，花费 $O(n)$ 时间。插入新键时要先找到它应当放在哪里，按键删除时也要先找到相应结点，所以这两个操作同样可能需要 $O(n)$ 时间。要提高效率，关键在于减少寻找位置时经过的结点数。

一个自然的办法是增加能跨过多个结点的指针。例如，在原链表上方，每隔两个结点连接一层指针，再每隔四个结点连接一层。查找时先沿高层指针跳过较长的一段，再逐层向下缩小查找范围，就不用从头逐个比较了。不过，如果始终要求各层严格保持这样的间隔，插入或删除一个结点后，后面许多结点的位置都会改变，可能需要调整许多指针。我们希望保留多层指针带来的查找效率，又不必在每次插入、删除后重新安排这些间隔。

跳表（skip list）用随机选择来决定哪些结点进入高层。每个结点都在第 1 层，然后独立抛公平硬币，正面就再增加一层，反面就停止。这样，结点进入第 h 层的概率为 $2^{-(h-1)}$，越高的层包含的结点越少。每一层仍按键值递增连接，但不再要求相邻结点之间恰好隔着相同数量的底层结点。记一个结点的层数随机变量为 H。若 $H \geqslant h$，需要前 $h-1$ 次都抛出正面，若 $H = h$，还需要下一次为反面，因此 $\Pr(H \geqslant h) = 2^{-(h-1)}$，$\Pr(H = h) = 2^{-h}$，$\mathbb{E}[H] = \sum_{h=1}^{\infty} 2^{-(h-1)} = 2$。图 13.4 展示了一个跳表的例子。

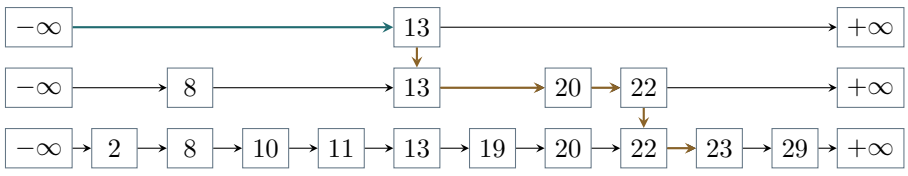


图 13.4　一张随机跳表。查找 23 时，先在第 3 层到 13，降至第 2 层到 22，再在底层定位到 23。

查找、插入与删除

跳表的这三种操作实现起来比较简单，下面只说明过程，不再列出伪代码，实现时注意指针的修改即可。沿用图中的表示，每层从头结点开始，以 NIL 结束，它们的键分别视为 $-\infty$ 和 $+\infty$。记当前最大层数为 L。

查找键 k 时，从最高层的头结点出发。如果下一结点的键小于 k，就向右走，否则从当前结点下降一层，继续用同样的方法查找。到底层后，如果下一结点的键恰好为 k，就找到了目标，如果下一结点的键大于 k，或者已经到了 NIL，就说明目标不存在。例如，在图 13.4 中查找 23，先在第 3 层走到 13。下一结点是 NIL，因此从 13 下降到第 2 层，经过 20 到达 22，再下降到底层，下一结点就是 23。

插入和删除也先按这个过程查找，同时记住在每一层停止向右移动时所在的结点（称为新结点在这一层的前驱）。它的键小于 k，而下一结点的键不小于 k。例如，要在图中插入 21，底层和第 2 层的前驱都是 20。插入新键时，按前面的抛硬币方法确定新结点的层数 H，再将它接入第 1 至第 H 层。在每一层中，先让新结点指向前驱原来指向的结点，再让前驱指向新结点。例如，插入一个层数为 2、键为 21 的结点，就是在底层和第 2 层分别将 $20 \to 22$ 改为 $20 \to 21 \to 22$。如果 $H > L$，就给头结点补上相应的高层指针，在这些新层中直接连接头结点、新结点和 NIL，并更新 L。如果查找时发现键已经存在，则更新原结点保存的值即可。删除时，若没有找到目标，无须修改，若找到了，就在目标所在的每一层，让前驱跳过目标，直接指向目标原来指向的结点，全部修改后再释放目标结点。例如，删除图中的 13 时，底层将 $11 \to 13 \to 19$ 改为 $11 \to 19$，第 2 层将 $8 \to 13 \to 20$ 改为 $8 \to 20$，第 3 层则改为头结点直接指向 NIL。此时第 3 层已经没有数据结点，可以删除这一层。一般地，删除后从最高层开始去掉空层，直到最高层非空或只剩底层，并相应更新 L。

期望复杂度

跳表的查找路径既有向右的移动，也有向下的移动。因此，要分析查找时间，既要知道跳表有多少层，也要知道在每层平均经过多少个结点。我们给定要保存的键和查找的目标，对抛硬币决定结点层数的过程取期望。

以下考虑 $n \geqslant 2$ 的情况，先估计最大层数 L。一个结点进入第 h 层的概率为 $2^{-(h-1)}$，而跳表至少有 h 层，就要求 n 个结点中至少有一个进入这一层。由并集界，$\Pr(L \geqslant h) \leqslant n2^{-(h-1)}$。取 $b = \lceil \log_2 n \rceil + 1$，则第 b 层的结点数期望为 $n2^{-(b-1)} \leqslant 1$，超过 b 的层数的期望满足 $\sum_{h=b+1}^{\infty} \Pr(L \geqslant h) \leqslant \sum_{h=b+1}^{\infty} n2^{-(h-1)} = n2^{-(b-1)} \leqslant 1$。也就是说，最大层数的期望至多为 $b + 1 = O(\log n)$。

接下来计算沿每层指针移动的步数。直接顺着查找过程分析不太方便，因为从哪一个结点下降，会受到上一层结构的影响。我们可以把已经走过的路径倒过来看，从底层停止查找的结点出发，逐步回到最高层的头结点。当前结点如果还有上一层，就向上走，否则向左走到本层前一个结点，直到找到能够上升的结点后上升。由于我们是通过独立抛公平硬币决定结点层数的，因此每次遇到一个结点时，它有上一层的概率都是 $1/2$。由引理 13.2，平均检查两个结点就能找到一个可以上升的结点，所以每层期望向左走一步，再向上走一步，合计两步。如果还没找到能上升的结点就先到了头结点，也可以直接上升，因此不会增加这个上界。

从底层走到第 b 层，按每层期望至多两步计算，总共至多为 $2(b-1)$ 步。对于第 b 层及更高的部分，向左经过的结点期望至多一个。此外，b 层起的期望上升也至多一步。由此，反向路径的期望长度为 $O(\log n)$，原来的查找路径与它一样长，查找时间也就是这一数量级。

插入和删除都先进行一次查找。插入时，新结点期望只有两层，因此查找之后，接入结点及必要

的增层只需期望 $O(1)$ 时间。删除时，修改指针和删除顶部空层至多处理 L 层，期望需要 $O(\log n)$ 时间。因此，三种操作的期望时间均为 $O(\log n)$。

跳表的空间复杂度也容易计算。由于每个结点在每层保存一个指针，并且期望层数为 2，所以 n 个结点期望共保存 $2n$ 个指针。此外，还有头结点占 $O(L)$ 空间，加起来期望空间仍为 $O(n)$。

13.8.3 素性检验

密码算法常需要选取很大的素数。我们可以先生成一个整数 N，再判断它是否为素数，但怎样才能快速完成这个判断？最直接的办法是依次试除 $2, \dots, \lfloor\sqrt{N}\rfloor$。如果 N 有 B 个二进制位，那么 $\sqrt{N} = \Theta(2^{B/2})$，试除次数可能随输入位数指数增长。因此我们希望结合素数的性质，设计更快的随机算法。下面介绍两种常用的素性检验方法。

费马检验

费马检验利用**费马小定理（Fermat's little theorem）**，通过检查一个整数是否满足素数应有的同余关系，来判断它是否为合数。

> **定理 13.5.** 设 p 是素数，a 是不被 p 整除的整数，则 $a^{p-1} \equiv 1 \pmod{p}$。

证明. 把 $1, \dots, p-1$ 分别乘以 a 后对 p 取余，得到的仍然是 $1, \dots, p-1$，只是顺序可能改变。这是因为若 $ai \equiv aj \pmod{p}$，就有 $p \mid a(i-j)$，由于 p 不整除 a，且 $1 \leqslant i, j < p$，只能有 $i = j$。因此，所得余数非零且互不相同，恰好将这 $p-1$ 个余数各取一次。因此 $a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$。由于 $(p-1)!$ 与 p 互素，可以约去，得到 $a^{p-1} \equiv 1 \pmod{p}$。 □

于是，对待检验的整数 $N > 2$，我们可以从 $1, \dots, N-1$ 中随机选一个底数 a，计算 $a^{N-1} \bmod N$。若结果不为 1，N 就违反了费马小定理，必定是合数。这就是费马检验。

如果结果为 1，则需要继续判断，因为有些合数也会满足这个同余式。例如，$341 = 11 \cdot 31$ 是合数，但 $2^{10} = 1024 \equiv 1 \pmod{341}$，从而 $2^{340} = (2^{10})^{34} \equiv 1 \pmod{341}$。因此，以 2 为底数时，费马检验会漏掉这个合数。

换一个底数有时能发现问题，但也有合数对所有与它互素的底数都满足费马同余式，这类数称为**卡迈克尔数（Carmichael number）**。例如 $561 = 3 \cdot 11 \cdot 17$，若 a 与 561 互素，它也分别与 $3, 11, 17$ 互素。由费马小定理，$a^2 \equiv 1 \pmod{3}$，$a^{10} \equiv 1 \pmod{11}$，$a^{16} \equiv 1 \pmod{17}$。因为 $560 = 2 \cdot 280 = 10 \cdot 56 = 16 \cdot 35$，将这三个同余式分别取 280、56、35 次幂，就得到 $a^{560} = (a^2)^{280} \equiv 1 \pmod{3}$，$a^{560} = (a^{10})^{56} \equiv 1 \pmod{11}$，$a^{560} = (a^{16})^{35} \equiv 1 \pmod{17}$。因此，$3, 11, 17$ 都整除 $a^{560} - 1$。它们两两互素，所以其乘积 561 也整除 $a^{560} - 1$，即 $a^{560} \equiv 1 \pmod{561}$。对于这样的数，只要抽到的底数与它互素，重复费马检验仍然发现不了问题，因此我们需要在计算过程中多检查一些素数必须满足的条件。

Miller–Rabin 检验

费马检验只检查最后算出的余数。我们还可以检查计算中间有没有出现素数不允许的情况。设 p 为素数，如果 $x^2 \equiv 1 \pmod{p}$，就有 $p \mid (x-1)(x+1)$，因此 p 必须整除其中一个因子，即 $x \equiv 1$ 或 $x \equiv -1 \pmod{p}$。对于待检验的整数 $N > 2$，我们考虑 $0 \leqslant x < N$ 中满足 $x^2 \equiv 1 \pmod{N}$ 的整数。1 和 $N-1$ 总满足这个条件，称为模 N 的 1 的平凡平方根。如果还能找到其他满足条件的 x，就称它

为非平凡平方根。前面已经证明，素数不会有这样的非平凡平方根，所以一旦找到它，就能确定 N 是合数。例如，$4^2 = 16 \equiv 1 \pmod{15}$，而 4 既不是 1 也不是 14，因此 4 是模 15 的 1 的非平凡平方根。

Miller–Rabin 检验把费马检验和非平凡平方根的检查结合起来。对待检验的整数 N，先处理简单情况：小于 2 的数不是素数，2 是素数，大于 2 的偶数都是合数。下面只考虑奇数 $N > 2$，首先将 $N-1$ 中的因子 2 全部提出，写成 $N-1 = 2^s d$，其中 d 为奇数。每轮从 $1, \ldots, N-1$ 中均匀抽取一个底数 a，先算出 $x_0 = a^d \bmod N$，再连续平方并取余，得到 $x_0, x_1, \ldots, x_s$，其中 $x_j = x_{j-1}^2 \bmod N = a^{2^j d} \bmod N$。最后一项 x_s 就是费马检验需要的 $a^{N-1} \bmod N$。计算过程中，每当得到 $x_j = 1$，就检查前一项 x_{j-1}。若前一项既不是 1 也不是 $N-1$，便找到了非平凡平方根，可以立即判定 N 为合数。若始终没有发现这种情况，再检查最后的 x_s：它不为 1 时，同样判定 N 为合数；它为 1 时，则本轮通过检验。

计算最初的 $a^d \bmod N$ 可以使用平方乘法。把 d 写成二进制，通过连续平方得到 $a, a^2, a^4, \ldots$，再把二进制位为 1 所对应的幂相乘。例如，$13 = 8 + 4 + 1$，所以 $a^{13} = a^8 a^4 a$。每次乘法后立即对 N 取余，最终就得到所需余数。指数 d 有多少个二进制位，就至多需要同样多次平方和同样多次乘法得到 a^d，因此共需 $O(\log d)$ 次模乘法。

如果一个底数没有发现问题，就重新独立抽取底数，再做一轮。我们预先规定检验 $t \geqslant 1$ 轮，只要有一轮出现合数的证据，就立即停止并判定为合数，如果全部通过，则报告可能是素数。素数一定能通过每轮检验，合数则可能漏判，后面将证明重复检验可以使漏判概率迅速减小。

例如，$N = 561$ 时有 $N - 1 = 2^4 \cdot 35$。取 $a = 7$，从 $7^{35} \bmod 561$ 开始连续平方，得到

$$241 \longrightarrow 298 \longrightarrow 166 \longrightarrow 67 \longrightarrow 1 \pmod{561}.$$

最后一步中，67 既不是 1，也不是 560，却满足 $67^2 \equiv 1 \pmod{561}$，因此算法能确定 561 是合数。

再看检验所需的时间。设 N 有 $B = \lfloor \log_2 N \rfloor + 1$ 个二进制位。计算 $a^d \bmod N$ 需要 $O(B)$ 次模乘法，后续又至多平方 B 次，所以每轮共需 $O(B)$ 次模乘法。每次参与运算的余数都小于 N，用普通大整数乘法和除法实现一次模乘法，需要 $O(B^2)$ 次位操作。因此，t 轮检验共需 $O(tB^3)$ 次位操作。此外，抽取底数还需要期望 $O(tB)$ 个随机位，因此不会改变总期望时间的数量级。

错误概率的证明

前面已经说明，素数一定通过每轮检验，但合数也可能通过检验。因此还需要回答：对于一个合数，随机选到的底数有多大概率能发现它是合数？下面证明，在 $1, \ldots, N-1$ 中，能使奇合数 N 通过检验的底数至多占一半。这样，每轮独立抽取底数，连续 t 轮都没有发现问题的概率就至多为 2^{-t}。

要限制通过检验的底数数量，我们先介绍一个计数方法。将 $1, \ldots, N-1$ 中与 N 互素的整数放入集合 $G = \{a : 1 \leqslant a < N,\ \gcd(a, N) = 1\}$，对任意 $a \in G$，由裴蜀定理，存在整数 u, v，使得 $ua + vN = \gcd(a, N) = 1$，所以 $ua \equiv 1 \pmod{N}$。将 u 除以 N 所得的余数记为 a^{-1}，它满足 $aa^{-1} \equiv 1 \pmod{N}$，称为 a 的乘法逆元。

假设我们找到一个集合 $B \subseteq G$，它包含 1，并且其中两个数相乘后对 N 取余，结果仍在 B 中，且其中每个数的逆元也在 B 中。如果还有一个 $w \in G$ 不属于 B，就把 B 中的每个数都乘以 w，再对 N 取余，将所得的数放入集合 wB。由于 w 有逆元，两个不同的数乘以 w 后仍然不同，所以 $|wB| = |B|$。而且 wB 与 B 不相交，否则就存在 $b_1, b_2 \in B$，使 $b_1 \equiv w b_2 \pmod{N}$。两边乘以 b_2^{-1}，可知 w 就是 $b_1 b_2^{-1}$ 除以 N 所得的余数，因而也在 B 中，与选择 w 的条件矛盾。因此，只要存在 $w \in G$ 不属于 B，G 就包含 B 和 wB 这两个大小相同、互不相交的子集，从而有 $2|B| \leqslant |G| \leqslant N - 1$。这个计数方法告诉我们，只要把全部通过检验的底数包含在一个这样的集合 B 中，再找到一个不属于 B 的 w，就能证明通过检验的底数至多有 $(N-1)/2$ 个。

定理 13.6. 对任意奇合数 N，在 $1,\dots,N-1$ 中，至多 $(N-1)/2$ 个底数能使 N 通过一轮 Miller–Rabin 检验。

证明. 若底数 a 使合数 N 通过检验，它首先必须通过最后的费马检查，即 $a^{N-1}\equiv 1 \pmod N$。因此存在整数 k，使得 $a\cdot a^{N-2}-kN=1$。如果一个正整数同时整除 a 和 N，它就整除等式左边的两项，因而也整除右边的 1，所以这个公因子只能是 1。这就说明 a 与 N 互素，即 $a\in G$。下面按照费马检验能否排除 G 中的某个底数，分两种情况讨论。

第一种情况，有某个 $w\in G$ 不能通过费马检验。取 $B=\{b\in G : b^{N-1}\equiv 1 \pmod N\}$，$B$ 就是 G 中所有能通过费马检验的底数，所以 $w\notin B$。要使用前面的计数方法，还需验证 B 包含 1，并且乘法取模与取逆元都不会得到 B 以外的数。首先，$1^{N-1}=1$，所以 $1\in B$。若 $b,c\in B$，则 $(bc)^{N-1}=b^{N-1}c^{N-1}\equiv 1\cdot 1=1 \pmod N$，因此 bc 对 N 取余后仍在 B 中。再看逆元。由于 $b\in B\subseteq G$，b 与 N 互素，前面的裴蜀定理已经保证它有逆元 b^{-1}，且 $bb^{-1}\equiv 1 \pmod N$。将这个同余式两边取 $N-1$ 次幂，得到 $b^{N-1}(b^{-1})^{N-1}\equiv 1 \pmod N$。其中 $b^{N-1}\equiv 1 \pmod N$，所以 $(b^{-1})^{N-1}\equiv 1 \pmod N$，即 b^{-1} 也在 B 中。这样，B 满足前面计数方法的全部条件，又有 $w\in G$ 不属于 B，便得到 $|B|\leqslant (N-1)/2$。通过 Miller–Rabin 检验至少需要通过费马检验，因此通过 Miller–Rabin 检验的底数都在 B 中，数量也就至多为 $(N-1)/2$。

第二种情况，G 中所有底数都通过费马检验。此时若仍按第一种情况定义 B，就会得到 $B=G$，无法找到集合以外的 w。因此要利用 Miller–Rabin 对中间平方结果的检查，重新选择 B。

仍写 $N-1=2^s d$，其中 d 是奇数。对每个 $a\in G$，考虑检验中依次算出的 $a^d \bmod N, a^{2d} \bmod N,\dots,a^{2^s d} \bmod N$，将这些项的下标依次记为 $0,1,\dots,s$。由于这一情况假设所有底数都通过费马检验，因此最后一项都是 1。若一个底数还能通过 Miller–Rabin 检验，那么它的序列要么从第一项起就全为 1，要么先出现 $N-1$，再一直为 1。这是因为，第一次得到 1 的前一项如果既不是 1 也不是 $N-1$，检验就发现了一个非平凡平方根，会判定 N 为合数。

这些序列出现 $N-1$ 的位置可能不同。我们把 G 中所有底数的序列放在一起看，找出 $N-1$ 最晚出现的下标 j，并选一个在第 j 项得到 $N-1$ 的底数 z。这样的下标一定存在，因为底数 $N-1$ 的第 0 项就是 $N-1$，又因为第 s 项全为 1，所以 $0\leqslant j<s$。记 $q=2^j d$，便有 $z^q\equiv -1 \pmod N$。取 $B=\{b\in G : b^q\equiv 1 \text{ 或 } -1 \pmod N\}$，对任何通过 Miller–Rabin 检验的底数，若它的序列出现过 $N-1$，这一项的下标至多为 j，所以第 j 项已经是 $N-1$ 或 1，若没有出现过 $N-1$，则各项都是 1。因此，所有通过检验的底数仍然都在 B 中。并且 B 也满足计数方法的三个条件，证明与第一种情况类似，不再赘述。

剩下要找一个 $w\in G$，使 w^q 模 N 后既不等于 1，也不等于 $N-1$。我们将把 N 写成两个互素因子 u,v 的乘积，让 w^q 模 u 时与 -1 同余，模 v 时却与 1 同余。这样，它就不可能模 N 时统一与 1 或 -1 同余。先说明 N 确实能拆成两个互素且大于 1 的因子。若不能，则合数 N 只能是某个素数的幂 p^e，其中 $e\geqslant 2$。取 $a=1+p^{e-1}$，它不被 p 整除，因而与 N 互素。二项式展开给出

$$a^{N-1}=(1+p^{e-1})^{p^e-1}\equiv 1+(p^e-1)p^{e-1}\equiv 1-p^{e-1}\not\equiv 1 \pmod{p^e}.$$

上式说明 $a\in G$ 却不能通过费马检验，与当前情况的假设矛盾。

现在使用之前选出的 z，它满足 $z^q\equiv -1 \pmod N$。由中国剩余定理，可以在 $0,\dots,N-1$ 中找到整数 w，使 $w\equiv z \pmod u$，$w\equiv 1 \pmod v$。因为 z 与 N 互素，它也与 u 互素，所以第一个同余式保证 w 与 u 互素，第二个同余式保证 w 与 v 互素。因此 w 与 $N=uv$ 互素，属于 G。另一方

面，将两个同余式取 q 次幂，得到 $w^q \equiv z^q \equiv -1 \pmod{u}$，$w^q \equiv 1 \pmod{v}$。若 $w^q \equiv 1 \pmod{N}$，就应有 $w^q \equiv 1 \pmod{u}$，与第一个同余式矛盾，若 $w^q \equiv -1 \pmod{N}$，就应有 $w^q \equiv -1 \pmod{v}$，与第二个同余式矛盾。这里 u, v 都是大于 1 的奇数，因此 1 与 -1 模它们均不同余。

这样就找到了 $w \in G$ 不属于 B。由前面的计数方法，$|B| \leqslant (N-1)/2$，而所有通过 Miller–Rabin 检验的底数都在 B 中，所以第二种情况也满足所需的数量上界，定理得证。 □

如果进一步细化计数，可以将单轮错误概率的上界改进为 $1/4$，得到常见的 4^{-t} 界，证明见参考资料。除了上述随机检验，素性判定也有确定性的多项式时间算法。Agrawal、Kayal、Saxena 在 2004 年发表的论文 *PRIMES is in P* 中给出了这样的算法。这说明，判断一个整数是否为素数，可以在关于其二进制位数的多项式时间内完成。

13.9 习题

习题 13.1 (期望时间与最坏时间)**.** 某个随机算法的期望运行时间为 $\Theta(n^2)$，是否仍可能在某次运行中花费 $\Omega(3^n)$ 时间？请构造具体的执行过程，并计算期望运行时间。

习题 13.2 (随机选择是否公平)**.** 1. 有人用下面的方法打乱三个互异元素：依次令 $i = 1, 2, 3$，每次都从全部三个位置中独立、均匀地选择 j，然后交换 $A[i]$ 与 $A[j]$。这样能得到均匀随机排列吗？

2. 一枚硬币每次抛出正面的概率为 $p \in (0, 1)$，各次抛掷相互独立，但我们不知道 p。如何只用这枚硬币，产生概率各为 $1/2$ 的 0 和 1？给出方法，并用 p 表示期望抛掷次数。

习题 13.3 (固定录用门槛)**.** 使用如下算法雇佣：前 k 位应聘者只参加面试，不予雇佣。此后，凡是比前 k 人中最优秀者更优秀的人，都予以雇佣，而且录用门槛始终不变。设 $1 \leqslant k \leqslant n$，各应聘者的能力互不相同，并以均匀随机顺序到来，求雇佣次数的期望。

习题 13.4 (不断加入新结点的网络)**.** 一个网络最初只有结点 v_1，此后 $v_2, \ldots, v_n$ 依次加入。每个新结点都从已有结点中均匀选择一个，向它连一条有向边，不同结点的选择相互独立。设 $n \geqslant 2$。

1. 求 v_j 最终入度的期望，并比较早加入和晚加入的结点的期望入度。
2. 求最终入度为 0 的结点数的期望。

习题 13.5 (选择互不冲突的任务)**.** 有 n 项任务需要执行，其中某些任务会使用同一资源，不能同时进行。用一个无向简单图表示这些冲突，每个顶点代表一项任务，相邻的任务不能同时执行。假设每项任务恰好与另外 $d \geqslant 1$ 项任务冲突，我们希望选出尽可能多的任务同时执行。

1. 每个顶点独立地以概率 p 标记为 1，否则标记为 0。只有自己标记为 1、所有邻居都标记为 0 的顶点才被选中。证明选出的任务互不冲突，求其数量的期望，并确定使期望最大的 p。
2. 将所有顶点均匀随机排列，只选出在排列中先于其所有邻居的顶点。证明这种方法也不会选出冲突的任务，求其数量的期望，并与第一问比较。

习题 13.6 (带权数据流抽样)**.** 数据流中的第 i 项数据 x_i 带有正整数权重 w_i。这次我们希望抽中某项的概率与其权重成正比，即处理完前 t 项后，第 i 项成为当前样本的概率为 $\frac{w_i}{w_1 + \cdots + w_t} (1 \leqslant i \leqslant t)$。

1. 每项数据只能读取一次，且只能保存一项样本和常数个辅助变量。请设计符合要求的抽样方法，并证明其正确性。

2. 两台机器分别处理互不重叠的两段数据，各自只保留一个符合上述要求的样本及该段的总权重。怎样利用这些信息，得到整个数据流的带权样本?

习题 13.7 (随机赋值求解 MAX SAT)**.** 给定布尔变量 $x_1, \dots, x_n$ 和子句 $C_1, \dots, C_m$。每个子句将若干变量或其否定用“或”连接，每个子句至少含一项，同一变量在一个子句中至多出现一次。子句 C_j 还带有非负权重 w_j，目标是给变量赋值，使所有为真的子句的权重之和尽可能大，记最优值为 OPT。

1. 让每个变量独立地以概率 $1/2$ 取真，证明所得总权重的期望至少为 $OPT/2$。若每个子句至少含三个变量，证明期望至少为 $7OPT/8$。
2. 假设只含一项的子句都形如 x_i，没有形如 $\neg x_i$ 的单项子句。让每个变量独立地以概率 $p \in [1/2, 1]$ 取真。请选取合适的 p，使所得总权重的期望至少为 $(\sqrt{5}-1)OPT/2$。

习题 13.8 (随机顶点覆盖)**.** 考虑顶点覆盖问题的如下随机算法。从空集 C 开始，每轮任取一条尚未被覆盖的边，以各 $1/2$ 的概率选择它的一个端点加入 C，并删去所有与所选顶点相连的边，直到没有边剩下。各轮选择端点时使用独立的随机结果。记最少需要的顶点数为 OPT。

1. 证明算法总能得到顶点覆盖，并且 $\mathbb{E}[|C|] \leqslant 2OPT$，即它是期望 2-近似算法。
2. 是否每次运行都能保证 $|C| \leqslant 2OPT$? 请证明或给出反例。

习题 13.9 (通过随机尝试寻找稠密子图)**.** 给定一个有 $n \geqslant 2$ 个顶点、$m \geqslant 1$ 条边的无向简单图，以及整数 $2 \leqslant k \leqslant n$。我们希望从中选出 k 个顶点，保留两端都被选中的边。请设计一个随机算法，使保留的边至少有

$$\mu = \frac{mk(k-1)}{n(n-1)}$$

条。算法返回的集合必须满足要求，期望运行时间须为多项式。

提示：先均匀随机选择 k 个顶点，计算保留边数的期望。如果失败就重新抽取，还需证明一次成功的概率不会太小，可以利用边数为整数这一点。

13.10 参考答案

习题 *13.1.* 可以。先执行 n^2 步，再独立抛 $2n$ 次公平硬币。只有全部为正面时，才额外执行 3^n 步。这个事件的概率为 4^{-n}，所以期望运行时间为 $\Theta(n^2) + O(n) + 4^{-n} \cdot 3^n = \Theta(n^2)$。全部为正面确实可能发生，此时运行时间就是 $\Omega(3^n)$。 □

习题 *13.2.* 1. 三次选择共有 $3^3 = 27$ 种等概率的结果，每种结果确定一个排列。如果六种排列等可能，每种就必须对应 $27/6$ 种选择结果，但这个数不是整数，因此不能得到均匀随机排列。

2. 每轮连续抛两次。先正后反时输出 0，先反后正时输出 1，两次相同时重新开始一轮。两种被接受结果的概率都是 $p(1-p)$，所以输出 0 和 1 的概率相同。每轮成功概率为 $2p(1-p)$，期望需要 $1/(2p(1-p))$ 轮，每轮两次，因此期望抛掷次数为 $1/(p(1-p))$。

□

习题 *13.3.* 对每个 $i > k$，第 i 人能被雇佣，当且仅当他比前 k 人都优秀。在这 $k+1$ 人中，每个人成为最优秀者的概率都是 $1/(k+1)$，所以期望雇佣次数为 $\sum_{i=k+1}^{n} \frac{1}{k+1} = \frac{n-k}{k+1}$。 □

习题 *13.4.* 1. 只有后来加入的结点可能向 v_j 连边。对每个 $i > j$，v_i 选择 v_j 的概率为 $1/(i-1)$，所以 $\mathbb{E}[\deg^-(v_j)] = \sum_{i=j+1}^{n} \frac{1}{i-1}$。对 $j < n$，这个和比 v_{j+1} 对应的和多一项 $1/j$，因此越早加入的结点，期望入度越大。

2. v_2 必须向 v_1 连边，所以 v_1 的入度不为 0。对 $2 \leqslant j \leqslant n$，要使 v_j 的入度为 0，所有后来加入的结点都必须避开它。由于选择独立，概率为 $\prod_{i=j+1}^{n} \left(1 - \frac{1}{i-1}\right) = \frac{j-1}{j} \frac{j}{j+1} \cdots \frac{n-2}{n-1} = \frac{j-1}{n-1}$。$j = n$ 时没有后来者，概率为 1，也符合最后的表达式。把各结点的概率相加，得到所求期望 $\sum_{j=2}^{n} \frac{j-1}{n-1} = \frac{n}{2}$。

□

习题 *13.5.* 1. 若一个顶点被选中，它的邻居都标记为 0，便不会被选中，所以选出的任务互不冲突。一个顶点要被选中，需要自己标记为 1，且 d 个邻居全部标记为 0，概率为 $p(1-p)^d$。选中数量的期望因此为 $np(1-p)^d$，求导可知在 $p = 1/(d+1)$ 取最大值 $\frac{n}{d+1}\left(\frac{d}{d+1}\right)^d$。

2. 相邻的两个顶点无法同时排在对方之前，所以不会同时被选中。对任意顶点，在它和它的 d 个邻居中，每个顶点最先出现的概率都是 $1/(d+1)$。因此选中数量的期望为 $n/(d+1)$，大于第一问能达到的最大期望。

□

习题 *13.6.* 1. 保存当前样本和已经读入的总权重 W，初始令 $W = 0$。读到 (x_i, w_i) 后，先令 $W \leftarrow W + w_i$，再从 $1, \ldots, W$ 中均匀取一个整数，若它不超过 w_i，就把样本换成 x_i。这样，新数据被选中的概率恰好为 w_i/W，第一项也一定被选中。用与第 13.6 节一样的方法可以证明结论。

2. 设两段总权重分别为 W_A, W_B。以概率 $W_A/(W_A + W_B)$ 返回第一段的样本，否则返回第二段的样本，并且这次选择独立于原来的抽样。如果某段为空，直接使用另一段的样本。

□

习题 *13.7.* 1. 一个含 ℓ 个变量的子句，只有其中每一项都为假才为假。各项使用不同变量，因此为假的概率为 $2^{-\ell}$。设全部子句权重之和为 W，显然 $OPT \leqslant W$。每个子句为真的概率至少为 $1/2$，所以总权重的期望至少为 $W/2 \geqslant OPT/2$。如果每个子句都至少含三项，为真的概率就至少为 $7/8$，同理得到 $7OPT/8$ 的保证。

2. 单项子句都形如 x_i，为真的概率为 p。其他子句至少含两项，每项为假的概率或者为 p，或者为 $1 - p \leqslant p$。由于不同项使用不同变量，整个子句为假的概率至多为 p^2。因此，每个子句为真的概率至少为 $\min\{p, 1-p^2\}$。两者相等时，它们的最小值最大，此时 $p = (\sqrt{5}-1)/2$，期望总权重至少为 $pW \geqslant (\sqrt{5}-1)OPT/2$。

□

习题 *13.8.* 1. 每条被删去的边都有一个端点加入了 C，所以算法结束时，C 覆盖了全部边。选过的顶点不再与任何剩余边相连，因而不会被重复选择，算法至多执行 $n = |V|$ 轮。

下面对图的顶点数归纳，证明 $\mathbb{E}[|C|] \leqslant 2OPT$。没有边时，算法不选任何顶点，结论成立。假设结论对顶点数更少的图成立，固定当前图的一个最优顶点覆盖 C^*。第一轮选出的边至少有一个端点属于 C^*，所以选中 C^* 内顶点的概率 $p \geqslant 1/2$。选中顶点后，将它及其相连的边从图中删去，后续算法就在剩余的图上继续执行。如果选中的顶点属于 C^*，那么 C^* 中剩余的 $OPT - 1$ 个顶点就能覆盖剩余的图，否则原来的 C^* 仍能覆盖剩余的图。剩余的图少了一个顶点，可以应用归

纳假设。因此，在上述两种情况下，后续选入的顶点数的期望分别至多为 $2(OPT-1)$ 和 $2OPT$。加上第一轮选中的一个顶点，得到

$$\mathbb{E}[|C|] \leqslant 1 + p \cdot 2(OPT-1) + (1-p) \cdot 2OPT = 2OPT + 1 - 2p \leqslant 2OPT.$$

2. 不能。取一个中心顶点与三个叶子相连的星形图，最优方案只需选择中心，故 $OPT = 1$。算法却可能连续三轮都选中叶子，得到 $|C| = 3 > 2OPT$，这种情况的概率为 $1/8$。

□

习题 *13.9.* 根据提示，每次均匀随机选出 k 个顶点，数出保留的边数 X。如果 $X \geqslant \mu$，就返回这个集合，否则独立地重新抽取。固定一条边，第一个端点被选中的概率为 k/n，在它已被选中的条件下，另一个端点被选中的概率为 $(k-1)/(n-1)$。因此，每条边被保留的概率为 $k(k-1)/(n(n-1))$，将 m 条边的指示变量相加，得到 $\mathbb{E}[X] = \mu$。还需估计一次成功的概率 p。令 $q = \lceil \mu \rceil$，由于边数为整数，失败时 $X \leqslant q-1$，成功时也至多为 m，所以 $\mu = \mathbb{E}[X] \leqslant (1-p)(q-1) + pm \leqslant q - 1 + pm$。于是 $p \geqslant (\mu - q + 1)/m$。这里 $\mu - q + 1 > 0$，而将这个数写成分母为 $n(n-1)$ 的分数后，分子是正整数，至少为 1。因此 $\mu - q + 1 \geqslant \frac{1}{n(n-1)}$，$p \geqslant \frac{1}{mn(n-1)}$，故期望尝试次数至多为 $mn(n-1)$。每次尝试先随机排列顶点，选取前 k 个，并扫描所有边来计数，需要 $O(n+m)$ 时间，故总期望时间为 $O(mn^2(n+m))$，是多项式时间。 □

13.11 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第十三讲。
2. Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein，算法导论，第三版，第 5、7、9 章及 31.8 节。可参阅随机排列、随机选择、快速排序与素性检验的分析。
3. Jon Kleinberg、Éva Tardos，*Algorithm Design*，2006，第 13 章。13.3、13.5、13.7 节分别讨论等待时间、随机分治和随机增量最近点对。
4. Mark Allen Weiss，数据结构与算法分析：*C* 语言描述，第二版，10.4 节。介绍伪随机数、跳表与素性检验。
5. William Pugh，*A Skip List Cookbook*，CS–TR–2286.1 ／ UMIACS–TR–89–72.1，1989，1990 年修订。除基本查找分析外，还讨论跳表的合并与分裂。
6. Pat Morin，*Closest Pair and Linear Programming*，COMP4804 讲义。介绍最近点对的随机增量算法与反向分析。
7. Alfred J. Menezes、Paul C. van Oorschot、Scott A. Vanstone，*Handbook of Applied Cryptography*，1996，4.2.3 节。给出 Miller–Rabin 单轮错误概率至多为 $1/4$ 的证明。
8. Manindra Agrawal、Neeraj Kayal、Nitin Saxena，*PRIMES is in P*，*Annals of Mathematics*，160(2):781–793，2004。证明素性判定可以在确定性多项式时间内完成。